Chat

SSE Chat

Start a Server-Sent Events (SSE) chat session with a specific agent snapshot

Overview

This endpoint initiates a Server-Sent Events (SSE) streaming chat session with a specific agent snapshot. The SSE protocol provides real-time, unidirectional communication from the server to the client.

Path Parameters

`agent_snapshot_id` `string` `required`

The unique identifier of the agent snapshot to chat with

Query Parameters

`end_user_id` `string` `required`

Identifier for the end user initiating the chat session

`chat_id` `string`

Optional chat ID to resume an existing chat session. If not provided, a new chat will be created.

`user_message` `string`

Message to pass to the agent

Authentication

This endpoint requires OAuth2 Bearer token authentication. Include the token in the Authorization header:

```
Authorization: Bearer your_access_token
```

Response

The endpoint returns a Server-Sent Events stream. Each event contains JSON data representing a `LeapingResponse` object. The responses are sent as data-only SSE events (no custom SSE event types).

Response Format

Each SSE event contains a JSON-serialized `LeapingResponse` with the following structure:

`type` `string` `required`

Discriminator field indicating the response type. Common values include:

- `chat_message` : Agent or human messages
- `turn` : Turn management (whose turn to speak)
- `error` : Error messages
- `transition` : Stage transitions in conversation flow
- `start` : Conversation initialization
- `end` : Conversation completion with summary and results
- `function` : Function call execution
- `kb` : Knowledge base lookup results

`timestamp` `number` `required`

Unix timestamp when the response was generated

Chat Message Response

For `type: "chat_message"` :

`sender` `string`

Message sender: `"human"` or `"bot"`

`text` `string`

The message content

Turn Response

For `type: "turn"` :

`turn` `string`

Current turn indicator: `"AGENT"` , `"USER"` , or `"END"`

Conversation End Response

For `type: "end"` :

`conversation_id` `string`

UUID of the conversation

`summary` `string | null`

AI-generated conversation summary

`results` `object`

Structured results extracted from the conversation

`fields` `object`

Final field values collected during the conversation

Error Response

For `type: "error"` :

`message` `string`

Error description

Usage Examples

JavaScript/Browser

```
const eventSource = new EventSource(
  '/chat/snapshot/your-snapshot-id?end_user_id=user123&user_message=Hello',
  {
    headers: {
      'Authorization': 'Bearer your_access_token'
    }
  }
```

··· See all 46 lines

Python with httpx-sse

```
import httpx
import json
import asyncio
from httpx_sse import aconnect_sse
from typing import Optional

class AgentSSEClient:
```

··· See all 182 lines

cURL

```
curl -X POST \
  "https://api.leaping.ai/v1/chat/snapshot/your-snapshot-id?end_user_id=user123" \
  -H "Authorization: Bearer your_access_token" \
  -H "Accept: text/event-stream"
```

Chat Flow

1. **Initialization**: The chat session starts with the specified agent snapshot
2. **Pre-conversation**: If the agent has pre-conversation stages, they execute first
3. **Turn Management**: The system manages turns between the agent and user
4. **Streaming Responses**: Agent responses are streamed in real-time via SSE
5. **Conversation End**: When the conversation concludes, final results are sent

Resuming Chats

To resume an existing chat session, include the `chat_id` parameter:

```
const eventSource = new EventSource(
  '/chat/snapshot/your-snapshot-id?end_user_id=user123&chat_id=existing-chat-id&user_message=Ok
);
```

Error Handling

Errors are sent as SSE events with `type: "error"` and include a timestamp:

```
{
  "type": "error",
  "message": "Agent not found",
  "timestamp": 1704067200.123
}
```

Common error scenarios:

- **404**: Agent snapshot not found
- **401**: Invalid or missing authentication token
- **403**: Insufficient permissions to access the agent
- **422**: Invalid parameters or chat state conflicts

Best Practices

- Always handle connection errors and implement reconnection logic
- Process different event types appropriately in your client
- Store the chat_id for the session resumption
- Implement proper authentication token refresh mechanisms
- Handle network interruptions gracefully